Decision trees are intuitive models that split data into branches based on feature values. The process involves recursive partitioning, where the dataset is divided into subsets that maximize information gain or minimize impurity (e.g., Gini impurity or entropy).

Advantages of Decision Trees:

- **Interpretability:** The decision-making process is visually represented, making it easy to understand.
- **Versatility:** They can handle numerical and categorical data.
- **No Need for Feature Scaling:** Unlike many algorithms, decision trees do not require feature scaling.

Decision Tree Implementation in Python

Below is an example of implementing a decision tree classifier using Scikit-Learn: from sklearn.tree import DecisionTreeClassifier from sklearn.model_selection import train_test_split from sklearn.metrics import accuracy_score import pandas as pd

```
# Example data
data = {
    'feature1': [2, 4, 6, 8, 10],
    'feature2': [1, 3, 5, 7, 9],
    'target': [0, 0, 1, 1, 1]
```

}

```
df = pd.DataFrame(data)
X = df[['feature1', 'feature2']]
y = df['target']

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42) # Train the model
decision_tree = DecisionTreeClassifier()
decision_tree.fit(X_train, y_train)

# Predictions
predictions = decision_tree.predict(X_test)

# Evaluation
accuracy = accuracy_score(y_test, predictions)
print("Accuracy:", accuracy)
```